arXiv:2406.12209v1 [cs.SD] 18 Jun 2024

Interface Design for Self-Supervised Speech Models

Yi-Jen Shih, David Harwath

Department of Computer Science, The University of Texas at Austin, USA

yjshih@utexas.edu, harwath@utexas.edu

Abstract

Self-supervised speech (SSL) models have recently become widely adopted for many downstream speech processing tasks. The general usage pattern is to employ SSL models as feature extractors, and then train a downstream prediction head to solve a specific task. However, different layers of SSL models have been shown to capture different types of information, and the methods of combining them are not well studied. To this end, we extend the general framework for SSL model utilization by proposing the interface that connects the upstream and downstream. Under this view, the dominant technique of combining features via a layerwise weighted sum can be regarded as a specific interface. We propose several alternative interface designs and demonstrate that the weighted sum interface is suboptimal for many tasks. In particular, we show that a convolutional interface whose depth scales logarithmically with the depth of the upstream model consistently outperforms many other interface designs.

Index Terms: self-supervised speech models, finetuning

1. Introduction

Speech processing tasks such as automatic speech recognition have traditionally relied on supervised learning algorithms that require transcribed training data. However, because transcriptions are expensive and time-consuming to collect, self-supervised speech models have recently become extremely popular. Speech models based on self-supervised learning (SSL) can be pre-trained with large amounts of unlabeled data, e.g. using a masked language modeling objective, and subsequently fine-tuned on a relatively small amount of labeled data for a target downstream task. Recent literature has proposed various SSL algorithms [1, 2, 3, 4] and many applications have been built on top of these SSL speech models [5], including Automatic Speech Recognition [6, 7], Visually Grounded Speech [8, 9, 10, 11], Speech Segmentation [9, 12] Emotion recognition [13], speaker verification [14] and so on. Furthermore, there are benchmarks proposed to evaluate speech SSL models on various downstream tasks [15, 16, 17, 18].

While significant progress has been made in developing algorithms and applications of speech SSL models, it is still unclear what the best method is for utilizing these models on downstream tasks. Existing methods for utilizing speech SSL models fall roughly into 3 categories: 1) Cascade the upstream model with a task-specific downstream prediction head and fine-tune the entire model. 2) Freeze the upstream model and take the output of a specific layer as the input features for a task-specific downstream module, then train the downstream module. 3) Compute a learnable weighted sum of all layers belonging to the upstream model and use the result as the input features for a task-specific downstream module. Typically, only the summation weights and the downstream module are trained during fine-tuning. Among the three methods, Fine-tuning the upstream and downstream models together often leads to better performance, but it is far more computationally expensive and can be unstable when a small amount of fine-tuning data is available. In contrast, using features from a single layer of the frozen upstream model is the most computationally inexpensive, however, it neglects the non-uniform distribution of different information types encoded across different layers of the upstream model [19]. The weighted sum can thus be viewed as an intermediate trade-off between the other two options and is the default configuration used in the SUPERB [15] benchmark.

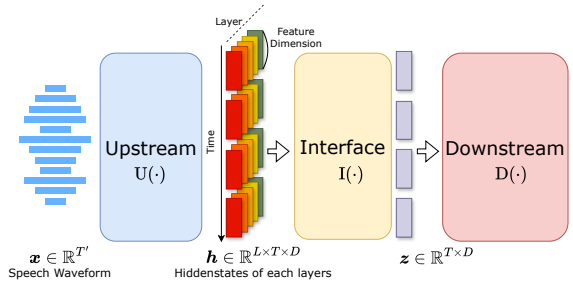

Figure 1: *The general framework for utilizing self-supervised speech models only considers the upstream and downstream model components. We argue that the interface connecting them should be considered in its own right as a separate component.(* L *: number of upstream model layers,* T *: upstream model output sequence length,* D *: upstream model feature dimension)*

However, in this paper, we argue that the weighted sum is still a suboptimal way of combining information across layers of speech SSL models. Our intuition is that because the information encoded in each dimension might differ across each layer of the upstream model, naively summing over the layers dimension-by-dimension may lead to an information loss in proportion to the degree of statistical independence between the same dimension across multiple layers.

Built upon this motivation, we hypothesize that there exists a better way of aggregating the information across different layers and introducing an **Interface** module in the SSL framework. The overall framework for utilizing speech SSL models then becomes a set of three components: the **Upstream** model, the **Downstream** prediction head, and the **Interface** that bridges between them by aggregating information across all upstream model layers. Under this definition, the widely-used weighted sum method is a specific type of interface, and in this paper, we

further propose several alternative interface designs. We evaluate the performance of each proposed interface across 5 upstream speech SSL models and several downstream tasks in the ML-SUPERB [18] and SUPERB [15] benchmarks. Among all our proposed interfaces, we show that Hierarchical Convolution across layers tends to achieve the best overall performance. We also conduct additional experiments showing that performance differences among various interface designs are due to their architectures themselves, and are not merely due to the difference in the number of trainable parameters. Additionally, we show that the Hierarchical Convolution interface still outperforms the weighted sum interface when the upstream model is fine-tuned with the downstream model. The code is publicly available. [1] Our contributions in this paper can be summarized as follows:

1. We propose an updated framework for self-supervised speech models that formally recognizes the interface module
2. We introduce several designs for the interface and show that the Hierarchical Convolution tends to perform the best across many speech processing tasks
3. We show that even when the full model is end-to-end fine-tuned, different interfaces still show performance differences.

2. Proposed Interface Methods

2.1. Interface Definition

We formalize our proposed overall pipeline for utilizing self-supervised speech models as a general framework (as shown in Fig. 1): **Upstream** $\rightarrow$ **Interface** $\rightarrow$ **Downstream**. The **Upstream** model $\mathrm{U}(\cdot) : \mathbb{R}^{T'} \mapsto \mathbb{R}^{L \times T \times D}$ stands for a pre-trained, self-supervised speech model which maps an utterance waveform $x \in \mathbb{R}^{T'}$ of length T' to a hidden representation $\boldsymbol{h}$ which consists of L representation layers each with a feature dimension of D. **Interface** is a function that aggregates information over the layer dimension L of the upstream output, $\mathrm{I}(\cdot) : \mathbb{R}^{L \times T \times D} \mapsto \mathbb{R}^{T \times D}$. Finally, **Downstream** $\mathrm{D}(\cdot)$ takes $\boldsymbol{z} = \mathrm{I}(\boldsymbol{h})$ as input and outputs predictions for a specific downstream task. Under this general framework, the weighted sum can be viewed as a specific type of interface,

$$\mathrm{I}_{\mathrm{WS}}(\boldsymbol{h}) = \sum_{l}^{L} w_l \cdot \boldsymbol{h}_l \tag{1}$$

where $\boldsymbol{w}$ is the per-layer weight vector, which is typically learned jointly with the downstream model weights.

The weighted sum interface has the benefit of being able to aggregate information across all L upstream layers with only L learnable parameters required. On the other hand, we hypothesize that it has potential downsides: by directly summing layer representations dimension-wise, statistically independent features from different layers of the upstream model may "collide" with one another resulting in information loss. As the number of layers in the upstream model increases, this problem can become more severe. To further investigate this, we propose several alternative interface designs that attempt to avoid information collision across multiple upstream layers while remaining relatively lightweight and also keeping the output dimension roughly the same as the upstream model's dimension.

[1] https://github.com/atosystem/SSL_Interface

2.2. Proposed Interfaces

2.2.1. Grouped Weighted Sums

An intuitive way to reduce the information collision of a single weighted sum is instead to use multiple weighted sums, each of which only has access to a subset of the upstream layers. Specifically, after conducting a weighted sum within each subset of upstream layers, we concat the results of each group along the feature dimension. Then we use a learnable projection to project the feature dimension to that of the downstream model.

2.2.2. Concatenation + Learnable Projection

Another type of interface concatenates all layer representations $\boldsymbol{h}$ along the feature dimension and uses a learnable projection layer to automatically decide the features that are beneficial to the downstream model. Specifically, we reshape $\boldsymbol{h}$ from (L, T, D) to $(T, L \times D)$, then project $(T, L \times D)$ back to the original upstream model dimension (T, D).

2.2.3. Hierarchical Convolution over Layers

Even though the feature manifold differs among each layer of the upstream model, neighboring layers can still possess similar feature distributions due to the residual connection in the transformer layers [20]. Motivated by the local structure of the hidden representation across the layer dimension, we apply 1D convolutions over the layer dimension to aggregate the information across layers. In practice, an interface design should be adaptable to upstream models with an arbitrary number of layers. Hence, in our paper, we fix the kernel size=5, stride=3 for our convolution layer and we stack $\lfloor \log_3 L \rfloor$ identical convolutional layers, which has the effect of collapsing the output of all models layers down to a single vector at the output of the convolutional stack (as shown in Fig. 2 (a)). We leave further convolutional hyperparameter optimization for future study.

2.2.4. CLS Pooling over layer dimension

We can use an attention module to aggregate information across layers rather than using convolutions. Specifically, at each timestep we view all the layers of the upstream model as a sequence of tokens and concatenate a learnable CLS vector [21] to this sequence. A Transformer layer is then used to aggregate information across all layers at the same time step into the CLS token (as shown in Fig. 2 (b)). A key difference compared to our other proposed interface designs is that CLS Pooling is data-dependent, while other methods are only task-dependent.

2.2.5. PCA + Concatenation

Besides the above parametric methods, we also investigated layer-wise dimensionality reduction via Principal Component Analysis (PCA), followed by concatenation across layers. Using the HuBERT Base model as an example upstream model, we take only the top 60 principal components of each Transformer layer and concatenate them together, resulting in $60 \times 13 = 780 \approx 768$ dimensions, which is roughly the same as the upstream model feature dimension. Practically, the PCA transform is learned from each downstream task dataset respectively.

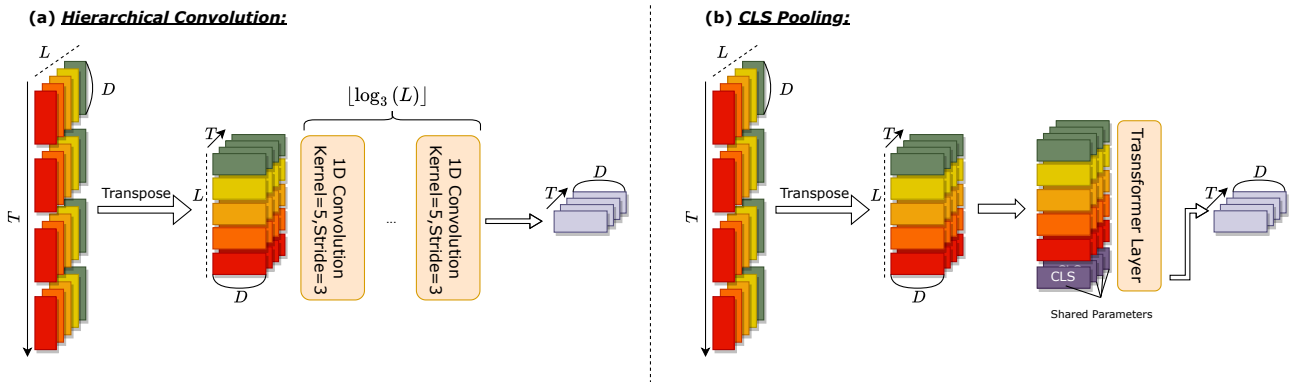

Figure 2: *Proposed Interface designs. (a): Hierarchical Convolution is applied over the layer dimension of the upstream model hiddenstates. (b): A learnable CLS embedding is used for summarizing over layer dimension*

Table 1: *Comparison between different proposed interfaces with HuBERT Base as the upstream model, evaluated on the ML-SUPERB Monolingual track. "GroupWS" indicates Grouped Weighted Sums and "Hierarchical Conv." stands for Hierarchical Convolution over layers. The metric is WER/CER.*

Interface	Params	Mono-10min	Mono-1hr
Weighted Sum	13	42.85	35.15
GroupWS(#Groups=2)	1.2M	41.84	34.47
GroupWS(#Groups=3)	1.8M	43.08	33.96
GroupWS(#Groups=4)	2.4M	42.52	33.99
Concat + Proj	7.7M	43.20	34.26
PCA + Concat	0	45.16	36.58
Hierarchical Conv.	4.4M	**41.51**	**33.88**
CLS Pooling	5.5M	48.36	33.92

3. Experiments

3.1. Experimental Setup

To get a holistic performance comparison of different interface designs, we choose 5 SSL models and 2 speech SSL benchmarks for evaluation. Specifically, we pick HubERT Base and Large [1], WavLM Base and Large [2], and XLSR-53 [3] for our upstream models. For downstream tasks, we test all proposed interfaces on ML-SUPERB [18] because it is currently less saturated than many SUPERB tasks. ML-SUPERB contains both 10min and 1hr training sets for each language. In the ML-SUPERB Monolingual track, the overall reported score is the average CER across 13 languages (the model for each language is trained individually). In the multilingual track, there are two tasks: **ASR** and Language Identification (**LID**). In these tasks, a model must be trained on 143 languages simultaneously.

In addition to multilingual ASR tasks, we also test the generalization of our interface designs on other common speech processing tasks. Hence, we select 4 tasks from SUPERB: Intent Classification (**IC**), Phoneme Recognition (**PR**), Emotion Recognition (**PR**), and Speaker Verification (**SV**).

To reduce the computational cost of running a large number of potential experiments, we first test all proposed interfaces on ML-SUPERB Monolingual Track with HuBERT Base. Then we pick the top 2 most promising interface designs and run them on all 5 upstream models and both ML-SUPERB and SUPERB. For both SUPERB and ML-SUPERB, we follow the default training configurations as far as possible. However, for the Multilingual track on ML-SUPERB, we find that the default downstream model is not optimal, and performance can be easily increased by simply adding more layers. In order to disentangle the effect of a larger downstream model from the interface itself, we do a hyperparameter search on the best number of layers in the downstream model so that it can achieve the lowest CER in terms of the same frozen upstream model. From our empirical experiment, we found that 65 is the optimal layer number for Multilingual ASR (about 56M params).

3.2. Interface Comparison on ML-SUPERB Monolingual ASR with HuBERT Base

The results for our pilot experiment using HuBERT Base on the ML-SUPERB Monolingual task are shown in Table 1. Firstly, we notice that there is a significant performance difference between all proposed interfaces relative to the baseline weighted sum method. While several of the proposed interfaces improve over the baseline weighted sum, we see that the Hierarchical Convolution achieves the overall best interface performance on both the 10min and 1hr datasets. The CLS Pooling interface ranks second on the 1hr task, but performs poorly on 10min, indicating that this method likely requires more finetuning data to reach its potential. In light of their promising results on the 1hr ML-SUPERB monolingual task, we choose Hierarchical Convolution and CLS Pooling for further experimentation.

3.3. Interface Comparison on ML-SUPERB and SUPERB

The results for the full suite of tasks across ML-SUPERB and SUPERB using all upstream models are shown in Table 2. Again, we observe that the Hierarchical Convolution interface tends to consistently outperform the baseline weighted sum as well as the CLS Pooling interface. The performance differences are often substantial: for example, simply replacing the weighted sum interface with the Hierarchical Convolution interface reduces HuBERT Base's PER on the SUPERB phone recognition task from 5.41 to 2.93, which is lower than the 3.53 PER achieved by HuBERT Large with the weighted sum interface. We even find that the CLS Pooling interface with the HuBERT Large model establishes a new SotA on the SUPERB leaderboard for phone recognition at 1.64 PER, beating the WavLM model's 3.06 PER when using the weighted sum.

Generally, the Hierarchical Convolution interface outperform the baseline weighted sum on all tasks except for Speaker Verification with HuBERT and WavLM models, although XLSR-53 with Hierarchical Convolution outperforms the weighted sum. This might imply that speaker-related infor-

Table 2: *Performance comparison of proposed interfaces using different upstream models, evaluated on both ML-SUPERB and SUPERB. The metric for each task: "Mono-1hr" and "Multi-1hr": WER/CER, "LID-1hr","ER","IC", "SV": accuracy, "PR": PER.*

Upstream	Interface	ML-SUPERB			SUPERB			
		Mono-1hr (↓)	Multi-1hr (↓)	LID-1hr (↑)	ER (↑)	IC (↑)	SV (↓)	PR (↓)
HuBERT Base	Weighted Sum	35.1	24.4	**84.6**	64.92	98.34	**5.11**	5.41
	Hierarchical Conv.	**33.9**	**24.0**	81.7	**68.49**	**99.45**	5.62	3.07
	CLS Pooling	**33.9**	24.3	1.2	62.73	**99.45**	49.91	**2.93**
HuBERT Large	Weighted Sum	32.3	22.3	64.4	67.62	98.76	**5.98**	3.53
	Hierarchical Conv.	**30.0**	**21.4**	**89.2**	**72.44**	**99.53**	6.03	1.76
	CLS Pooling	31.1	21.5	87.0	68.72	**99.53**	6.34	**1.64**
WavLM Base	Weighted Sum	34.2	24.3	**84.1**	65.94	98.63	**4.69**	4.84
	Hierarchical Conv.	**32.4**	**23.6**	67.9	**68.57**	**99.53**	5.48	3.06
	CLS Pooling	33.8	23.8	13.2	63.60	99.50	6.03	**2.92**
WavLM Large	Weighted Sum	30.1	20.8	71.7	70.62	99.31	**3.77**	3.06
	Hierarchical Conv.	**28.0**	19.4	**90.6**	**74.95**	**99.71**	5.20	**1.72**
	CLS Pooling	29.7	**18.8**	88.5	70.45	99.63	4.27	1.73
XLSR-53	Weighted Sum	35.1	20.2	76.6	66.34	95.62	6.45	4.50
	Hierarchical Conv.	**30.6**	**19.8**	**80.4**	**72.01**	**99.55**	**5.63**	**2.69**
	CLS Pooling	31.0	80.7	4.5	65.10	99.47	12.95	5.27

mation is encoded similarly across different layers in HuBERT and WavLM, causing our proposed interfaces to overfit.

Interestingly, in both benchmarks, we also observe a larger improvement over the basic weighted sum in Large models compared to Base models for HuBERT and WavLM when Hierarchical Convolution is applied, which supports our hypothesis that the basic weighted sum averages out more information as the number of layers increases. In particular, on the LID task, we see that with Base upstream models the Hierarchical Convolution interface underperforms the basic weighted sum, but when scaling to the Large models (HuBERT Large, WavLM Large, and XLSR-53) the Hierarchical Convolution instead outperforms the basic weighted sum.

3.4. Interface vs. Larger Downstream Models

Table 3: *Ablation experiments on trainable parameters. "WS w/ Large DS" indicates Weighted Sum with Large Downstream model.*

Upstream	Interface	ML-SUPERB (WER/CER)	
		Mono-1hr (↓)	Multi-1hr (↓)
HuBERT Base	Hierarchical Conv.	**33.9**	**24.0**
	WS w/ Large DS	35.2	24.8
HuBERT Large	Hierarchical Conv.	**30.0**	**21.4**
	WS w/ Large DS	32.9	22.1
WavLM Base	Hierarchical Conv.	**32.4**	**23.6**
	WS w/ Large DS	34.4	24.3
WavLM Large	Hierarchical Conv.	**28.0**	**19.4**
	WS w/ Large DS	30.5	20.8
XLSR-53	Hierarchical Conv.	**30.6**	**19.8**
	WS w/ Large DS	35.4	20.0

Although there are substantial improvements using the Hierarchical Convolution interface, it is not immediately clear whether the performance gain comes from the interface design or simply the additional trainable parameters. To this end, we increase the size of the downstream model for Monolingual and Multilingual ASR on ML-SUPERB by an amount roughly the same as the number of parameters in the Hierarchical Convolution interface (denoted as "WS+Large DS").

As shown in Table 3, under all settings, weighted sum with a large downstream model does not lead to better performance than the Hierarchical Convolution interface combined with a smaller downstream model. This implies the role of the interface is not simply to add more learnable parameters to the downstream model, but to actually better aggregate information across the layers of the upstream model. Our findings not only resonate with [22] that larger downstream models increase the performance but we also show that a better interface design can get further improvements.

3.5. Performance under Fine-tuning setting

We motivated the introduction of the interface module under the assumption that the upstream model would typically remain frozen during the entire downstream task fine-tuning. However, we wish to empirically verify whether different interfaces can still offer improvements with end-to-end fine-tuning. Practically, we follow the same pipeline as in our previous experiments except the upstream model is trainable. We evaluate on ML-SUPERB Monolingual track 1h using HuBERT Base.

Table 4: *Comparison of Weighted Sum and Hierarchical under end-to-end fine-tuning. The metric is WER/CER.*

Interface	ML-SUPERB Mono-1hr (↓)	
	Freeze Upstream	Fine-tuned
Weighted Sum	35.1	31.5
Hierarchical Conv.	**33.9**	**31.1**
WS w/ Large DS	35.2	31.6

As shown in Table 4, end-to-end fine-tuning shortens the gap between different interface designs. However, the Hierarchical Convolution interface still outperforms the weighted sum baseline. These results indicate that the interface design is beneficial regardless of frozen or trainable upstream model.

4. Conclusion

In this work, we introduced the interface module into the conceptual framework for utilizing self-supervised speech models.

We also proposed several new interface designs. From our experiments on multiple benchmarks and upstream models, we showed that the Hierarchical Convolution interface tends to outperform our other proposed designs, as well as the weighted sum method that is currently standard practice in the field. Finally, our ablation experiments substantiate that the role of the interface in optimally aggregating information across layers of the upstream model is more important than the amount of trainable parameters during funetuning.

5. Acknowledgements

This work is supported in part by the National Science Foundation under Grant No. 2238605.

6. References

[1] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, "Hubert: Self-supervised speech representation learning by masked prediction of hidden units," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 3451–3460, 2021.

[2] S. Chen, C. Wang, Z. Chen, Y. Wu, S. Liu, Z. Chen, J. Li, N. Kanda, T. Yoshioka, X. Xiao, J. Wu, L. Zhou, S. Ren, Y. Qian, Y. Qian, M. Zeng, and F. Wei, "Wavlm: Large-scale self-supervised pre-training for full stack speech processing," *IEEE Journal of Selected Topics in Signal Processing*, vol. 16, pp. 1505–1518, 2021.

[3] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, "Unsupervised cross-lingual representation learning at scale," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Jul. 2020, pp. 8440–8451.

[4] S. Schneider, A. Baevski, R. Collobert, and M. Auli, "wav2vec: Unsupervised pre-training for speech recognition," in *Interspeech*, 2019.

[5] A. Mohamed, H.-y. Lee, L. Borgholt, J. D. Havtorn, J. Edin, C. Igel, K. Kirchhoff, S.-W. Li, K. Livescu, L. Maaløe *et al.*, "Self-supervised speech representation learning: A review," *IEEE Journal of Selected Topics in Signal Processing*, 2022.

[6] A. Baevski, W.-N. Hsu, A. Conneau, and M. Auli, "Unsupervised speech recognition," *NeurIPS*, 2021.

[7] A. H. Liu, W.-N. Hsu, M. Auli, and A. Baevski, "Towards end-to-end unsupervised speech recognition," in *2022 IEEE Spoken Language Technology Workshop (SLT)*, 2023, pp. 221–228.

[8] P. Peng and D. Harwath, "Fast-slow transformer for visually grounding speech," in *ICASSP*, 2022.

[9] ——, "Word discovery in visually grounded, self-supervised speech models," in *Interspeech*, 2022.

[10] Y.-J. Shih, H.-F. Wang, H.-J. Chang, L. Berry, H.-y. Lee, and D. Harwath, "Speechclip: Integrating speech with pre-trained vision and language model," in *2022 IEEE Spoken Language Technology Workshop (SLT)*, 2023, pp. 715–722.

[11] L. Berry, Y.-J. Shih, H.-F. Wang, H.-J. Chang, H.-Y. Lee, and D. Harwath, "M-speechclip: Leveraging large-scale, pre-trained models for multilingual speech to image retrieval," in *ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023, pp. 1–5.

[12] L. Strgar and D. Harwath, "Phoneme segmentation using self-supervised speech models," in *2022 IEEE Spoken Language Technology Workshop (SLT)*, 2023, pp. 1067–1073.

[13] E. Morais, R. Hoory, W. Zhu, I. Gat, M. Damasceno, and H. Aronowitz, "Speech emotion recognition using self-supervised features," in *ICASSP 2022 - 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022, pp. 6922–6926.

[14] J. Peng, T. Stafylakis, R. Gu, O. Plchot, L. Mošner, L. Burget, and J. Černocký, "Parameter-efficient transfer learning of pre-trained transformer models for speaker verification using adapters," in *ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023, pp. 1–5.

[15] S. Yang, P.-H. Chi, Y.-S. Chuang, C.-I. J. Lai, K. Lakhotia, Y. Y. Lin, A. T. Liu, J. Shi, X. Chang, G.-T. Lin, T.-H. Huang, W.-C. Tseng, K. tik Lee, D.-R. Liu, Z. Huang, S. Dong, S.-W. Li, S. Watanabe, A. Mohamed, and H. yi Lee, "SUPERB: Speech Processing Universal PERformance Benchmark," in *Proc. Interspeech 2021*, 2021, pp. 1194–1198.

[16] H.-S. Tsai, H.-J. Chang, W.-C. Huang, Z. Huang, K. Lakhotia, S.-w. Yang, S. Dong, A. Liu, C.-I. Lai, J. Shi, X. Chang, P. Hall, H.-J. Chen, S.-W. Li, S. Watanabe, A. Mohamed, and H.-y. Lee, "SUPERB-SG: Enhanced speech processing universal PERformance benchmark for semantic and generative capabilities," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, S. Muresan, P. Nakov, and A. Villavicencio, Eds. Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 8479–8492. [Online]. Available: https://aclanthology.org/2022.acl-long.580

[17] S. Shon, S. Arora, C.-J. Lin, A. Pasad, F. Wu, R. S. Sharma, W.-L. Wu, H.-y. Lee, K. Livescu, and S. Watanabe, "SLUE phase-2: A benchmark suite of diverse spoken language understanding tasks," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Jul. 2023, pp. 8906–8937. [Online]. Available: https://aclanthology.org/2023.acl-long.496

[18] J. Shi, D. Berrebbi, W. Chen, E.-P. Hu, W.-P. Huang, H.-L. Chung, X. Chang, S.-W. Li, A. Mohamed, H. yi Lee, and S. Watanabe, "ML-SUPERB: Multilingual Speech Universal PERformance Benchmark," in *Proc. INTERSPEECH 2023*, 2023, pp. 884–888.

[19] A. Pasad, B. Shi, and K. Livescu, "Comparative layer-wise analysis of self-supervised speech models," in *ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023, pp. 1–5.

[20] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *NeurIPS*, 2017.

[21] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds. Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186. [Online]. Available: https://aclanthology.org/N19-1423

[22] S. Zaiem, Y. Kemiche, T. Parcollet, S. Essid, and M. Ravanelli, "Speech Self-Supervised Representation Benchmarking: Are We Doing it Right?" in *Proc. INTERSPEECH 2023*, 2023, pp. 2873–2877.